

Proyecto Página Web Ciberseguridad

CyberIncident – Mejoras Unidad 2

Grupo 7 - Kevin Marcillo
Andrés Sierra
Ricardo Simbaña
Fernando Vaca

12 de septiembre de 2026

Evidencias de Funcionamiento

A continuación, se presentan las evidencias correspondientes al funcionamiento del prototipo mejorado de CyberIncident. Las capturas muestran la carga de información mediante Fetch API, la interacción con los incidentes, la validación dinámica del formulario, la actualización de los datos y la adaptación responsive de la interfaz.

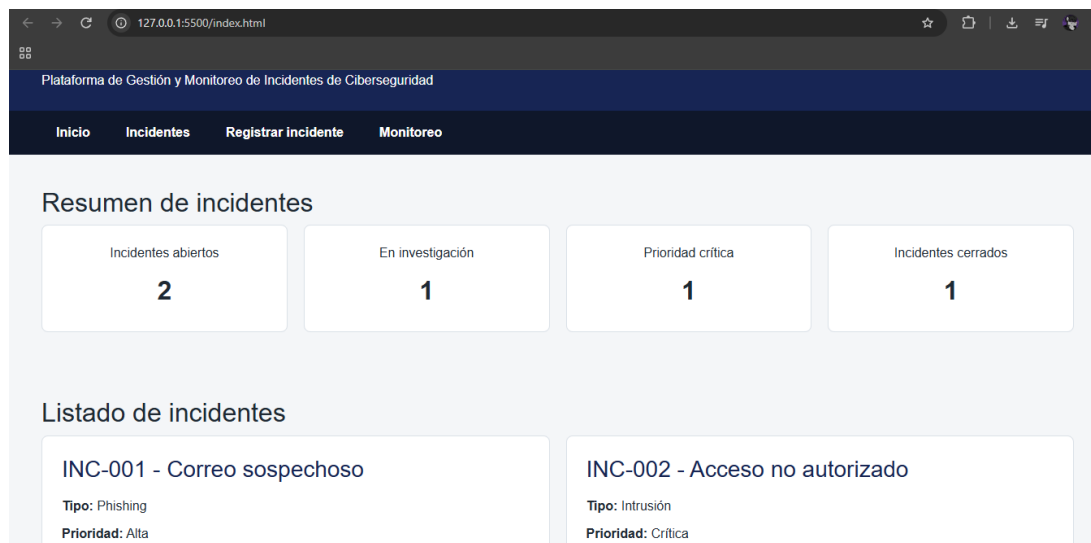


Figura 1: Inicio de la página web, corriendo en Live Server.



Figura 2: Listado de incidentes (Fetch a Json) y visualización del detalle funcional.

The figure shows a browser window displaying a form titled 'Registrar nuevo incidente'. The form includes the following sections and validation messages:

- Complete todos los campos obligatorios para registrar un nuevo incidente de ciberseguridad.**
- Título del incidente:**
 - Input field with 'sub'.
 - Message: 'Escriba un título breve y descriptivo, entre 5 y 100 caracteres.'
 - Message: 'El título debe tener al menos 5 caracteres.'
- Tipo de incidente:**
 - Dropdown menu with 'Seleccione un tipo'.
 - Message: 'Seleccione la categoría que mejor describa el incidente.'
 - Message: 'Debe seleccionar un tipo de incidente.'
- Prioridad del incidente:**
 - Radio buttons for 'Baja', 'Media', 'Alta', and 'Critica'.
 - Message: 'Debe seleccionar una prioridad.'

Figura 3: Formulario con validación dinámica y mensajes de error accesibles.



127.0.0.1:5500/index.html

Introduzca un correo electrónico válido para identificar al reportante.

Fecha del incidente

dd/mm/aaaa

Seleccione la fecha en la que ocurrió o fue detectado el incidente.

Descripción del incidente

Describa brevemente qué ocurrió. Utilice entre 20 y 500 caracteres.

Registrar incidente

Incidente registrado exitosamente

INC-005 - Subida sospechosa de archivos

Tipo: Intrusión
Prioridad: Crítica
Estado: Abierto
Fecha: 2026-09-12

Ver detalle

Detalle del incidente

INC-005 - Subida sospechosa de archivos

Tipo: Intrusión
Prioridad: Crítica
Estado: Abierto
Fecha del incidente: 2026-09-12
Reportado por: kevcillo16@gmail.com
Responsable: Equipo de Seguridad

Descripción

Se detectó una subida sospechosa de archivos, no suele ser el tipo de archivo usual

Última actualización: 2026-09-12

Figura 4: Registro exitoso de un nuevo incidente, mensaje de confirmación y actualización del resumen.



Figura 5: Versión móvil responsive con menú de navegación tipo hamburguesa.

Justificación técnica

El proyecto **CyberIncident** corresponde a la segunda versión de una interfaz web para la gestión y monitoreo de incidentes de ciberseguridad, desarrollada como continuación del prototipo de la Unidad 1. Esta entrega incorpora JavaScript moderno, manipulación del DOM, Fetch API, accesibilidad con ARIA, jQuery y una separación de responsabilidades mediante el patrón Modelo–Vista–Controlador (MVC).

En cuanto a **JavaScript moderno**, se reemplazó la lógica anterior por clases ES6 (`IncidentModel`, `IncidentView` e `IncidentController`), funciones flecha, `const/let`, plantillas de cadenas y funciones asíncronas mediante `async/await`. El código se distribuyó en módulos independientes

(`model.js`, `view.js`, `controller.js` y `app.js`), utilizando `import/export` y `<script type="module">`. Esta organización mejora la mantenibilidad del proyecto y separa sus responsabilidades.

El **modelo local de solo lectura** obtiene los incidentes base desde `data/incidentes.json` mediante **Fetch API**. La solicitud utiliza `async/await` y `try/catch` para controlar errores de carga. Los datos originales se protegen mediante `Object.freeze()`, mientras que los nuevos incidentes registrados por el usuario se almacenan temporalmente en un arreglo independiente en memoria. Debido al uso de Fetch sobre un archivo local, la aplicación debe ejecutarse mediante un servidor HTTP local y no directamente con `file://`.

La **manipulación del DOM** se realiza dinámicamente a partir de los datos obtenidos. El listado, el resumen y el detalle de los incidentes se generan durante la ejecución. Para estas operaciones se utiliza **jQuery**, aprovechando su selección de elementos, creación de contenido y manejo de eventos. Además, los textos ingresados por el usuario son procesados mediante una función de escape antes de incorporarse al DOM, reduciendo el riesgo de ataques de *Cross-Site Scripting* (XSS).

En materia de **accesibilidad**, se incorporó una región `aria-live="polite"` para anunciar cambios de estado, como cargas, errores y confirmaciones. Al registrar correctamente un incidente, la Vista muestra el mensaje visible “Incidente registrado exitosamente” y el Controlador también anuncia mediante ARIA que el incidente fue registrado correctamente. Los campos del formulario utilizan `aria-invalid`, `aria-describedby` y elementos `role="alert"` para comunicar los errores. El grupo de prioridades utiliza `fieldset` y `legend`. Cuando existen errores, el foco se desplaza al primer campo que requiere corrección, facilitando la navegación mediante teclado y reforzando los criterios de las WCAG.

La **validación dinámica** complementa los atributos HTML5 como `required`, `minlength`, `maxlength` y `type="email"`. Las reglas se ejecutan mediante JavaScript ante eventos como `input`, `blur` y `change`, permitiendo mostrar mensajes específicos para cada campo. El formulario utiliza `novalidate` para que los mensajes personalizados sean controlados por la aplicación.

Como mejora de la experiencia responsive, se incorporó un **menú de navegación tipo hamburguesa** para dispositivos móviles. Mediante jQuery se controla el botón y se alterna la clase `menu-abierto`, mientras que `aria-expanded` y `aria-controls` comunican el estado y la relación del menú con la navegación. El menú se cierra al seleccionar una opción o mediante la tecla Escape, devolviendo el foco al botón. En CSS, esta navegación se activa únicamente en pantallas de hasta 600px, manteniendo la navegación convencional en pantallas mayores.

Finalmente, la arquitectura **MVC** separa las responsabilidades del sistema. El **Modelo** administra los datos y su carga; la **Vista** se encarga del DOM, jQuery y los elementos de accesibilidad; y el **Controlador** coordina los eventos, la validación y la comunicación entre ambos componentes. Esta estructura facilita el mantenimiento del código y permite una futura ampliación hacia un backend y una base de datos reales.

En conjunto, las mejoras permiten que CyberIncident pase de un prototipo inicial a una interfaz más dinámica, organizada, accesible y adaptable, aplicando los principales contenidos de la Unidad 2.